

Milestone 4. Frontend Development

Activity 4.1. Designing and Developing User Interface

Activity 4.2. Creating Dynamic Templates with FastAPI's Jinja2

Milestone 4. Frontend Development

Milestone 4 focuses on developing a user-friendly and visually structured interface for FitBuddy, the AI-powered workout generator. This involves building responsive and interactive web pages using HTML, CSS, and Jinja2 templating. Each template is dynamically rendered through FastAPI, ensuring users can easily submit input, receive personalized plans, provide feedback, and allow administrators to manage data effectively. This milestone guarantees an engaging and intuitive fitness planning experience for all users.

Activity 4.1 Designing and Developing the User Interface

Set Up the Base HTML Structure

Developed index.html as the main entry point for user input.

Structured the form to capture:

username, user_id, age, weight, goal, and intensity.

Each field is properly labeled and grouped using semantic HTML.

The design uses a gym-themed background image, Google Fonts (Roboto), and bold typography to create a clean, fitness-oriented interface.

Navigation is kept minimal for focus, and user flow directs clearly from input to output.

Design a Responsive Layout Using CSS

Embedded CSS styles were applied directly in each HTML file.

Layouts use Flexbox to center content both vertically and horizontally.

Media queries ensure mobile responsiveness.

Buttons have consistent styling with hover effects.

Input fields and result blocks have adequate padding, shadows, and rounded corners for modern aesthetics.

Overall color scheme supports readability on a dark background.

Create Separate Pages for Each Core Functionality

Three Jinja2-powered HTML templates were created under /templates:

index.html: A user input form with fields for fitness goal, intensity, and basic personal information.

result.html: Displays the 7-day AI generated workout plan, a nutrition tip, and accepts user feedback.

all_users.html: Admin panel to view and delete users and review both original and updated plans.

Templates are modular, and UI transitions smoothly between steps.

Forms are submitted to specific FastAPI endpoints via POST methods

Activity 4.2 Creating Dynamic Templates with FastAPI's Jinja2

Integrate Jinja2 Templating for Dynamic Content Rendering

Used FastAPI's Jinja2Templates for template rendering:

```
from fastapi.templating import Jinja2Templates

BASE_DIR = os.path.dirname(os.path.abspath(__file__))

TEMPLATE_DIR = os.path.join(BASE_DIR, "templates")

templates = Jinja2Templates(directory=TEMPLATE_DIR)
```

Each route uses TemplateResponse to render the proper HTML with passed context:

```
Return templates.TemplateResponse("result.html", {

    "request": request,

    "username": username,
```

```
    "user_id": user_id,  
    "age": age,  
    "weight": weight,  
    "goal": goal,  
    "intensity": intensity,  
    "workout_plan": plan,  
    "nutrition_tip": nutrition_tip  
})
```

This allows dynamic plan and nutrition tip display after form submission.

Bind Backend Data to HTML Templates

► index.html

Form elements use name=".." to bind to backend Form(...) parameters.

Submits data to /generate-workout.

► result.html

Uses:

{{ username }}, {{ goal }}, {{ intensity }}, etc. to show user data.

{{ workout_plan }} and {{ nutrition_tip }} to show AI-generated results.

Includes a feedback form submitted via POST to /submit-feedback.

► all_users.html

Loops over all user records using:

HTML

</tr>

{% for user in users %}

<tr>

<td>{{ user.id }}</td>

<td>{{ user.name }}</td>

<td>{{ user.age }}</td>

<td>{{ user.weight }}</td>

<td>{{ user.goal }}</td>

<td>{{ user.intensity }}</td>

<td><pre>{{ user.original_plan }}</pre></td>

<td><pre>{{ user.updated_plan }}</pre></td>